

MOFSL client onboarding

Application and code walkthrough

ASP.NET Core 10 REST API / React 19 / SQLite

8 October 2026 / Branch final-demo

Follow a client request from sign-in to account opening, then trace the save and reporting code. This PDF covers all 39 segments of the verified HTML guide. Each segment names the source files and line ranges that implement it.

The interactive companion is [documents/client-onboarding-flow.html](#). Its sidebar contains full verified source excerpts. This PDF is a static reading copy.

Source basis

The received onboarding Word document, agreed project decisions, current code, and [documents/08-specification-audit.md](#). This walkthrough reflects the working tree. Changes on final-demo have not been committed or pushed.

Reading path

- New user: read setup and Follow the six stages.
- Developer: read Trace one save and Read and maintain the code.
- Manager: read Understand the MIS and the limits at the end.

Setup and reading map

Run from scratch

Install Node.js 22.12+ and the .NET 10 SDK. Run these commands from the project directory. The API serves the built React application.

```
npm install
npm run build
npm start
```

Open <http://127.0.0.1:4173/>. For React hot reload, keep the API running, run `npm run dev` separately, and open <http://127.0.0.1:5173/>. Demo credentials for each role are listed in README.md. Replace them before shared deployment.

The request path

Browser action -> React form state -> typed `api()` helper -> session middleware -> API endpoint -> application service -> SQLite transaction -> JSON response -> refreshed workspace.

Case, entry, and history

A case has one stable Reference ID. Related entries share that ID. Status history records append-only changes with the authenticated actor. An edit counts as a touch even when its status stays the same.

Contents

1. Start from scratch
2. Enter the workspace
3. Follow the six stages
4. Trace one save
5. Understand the MIS
6. Upload and administer
7. Read and maintain the code

Verification and limits

Start from scratch

01. What runs in the browser and on the server?

React displays screens. ASP.NET Core accepts requests. SQLite stores the register.

User actions

- Open the application in a browser. The browser loads HTML, CSS, and a JavaScript bundle.
- Use the forms and tables. React updates the displayed page as its state changes.

Code flow

- TypeScript checks types during development; Vite compiles and bundles it for the browser.
- The workspace hook calls /api endpoints using JSON and the session cookie.
- The C# API validates permissions, writes SQLite, and calculates MIS. React displays the returned numbers.

Data and rules

Browser -> api() -> API middleware -> endpoint -> focused service -> SQLite -> JSON -> React state.

The application contains only the ASP.NET Core API and React frontend. MIS calculations remain on the server.

Source blocks

backend/dotnet/Program.cs / lines 4-28 / var builder
frontend/lib.ts / lines 112-125 / export async function api
frontend/app.tsx / lines 27-240 / function App()

02. A case, an entry, and an event are different records

One stable Reference ID can have many operational entries and status-history events.

User actions

- Create a New entry for the original request.
- Use its Reference ID for a resubmission, discrepancy resolution, or modification.

Code flow

- cases stores shared client metadata such as PAN, channel, location, and CSE.
- case_entries stores request IDs, process dates, owner, and the status of each operational row.
- status_history stores actual status changes and the authenticated actor. Foreign keys connect these records.

Data and rules

cases.id -> case_entries.case_id; cases.id and case_entries.id -> status_history.case_id / entry_id.

Adding a related entry can update shared client metadata. Reports count cases, while the register lists entries.

Source blocks

sql/schema.sql / lines 56-122 / CREATE TABLE IF NOT EXISTS cases
backend/dotnet/CaseQueries.cs / lines 8-20 / private const string CaseSql

03. Install, build, and start locally

Build the React assets, start the API, then open localhost.

User actions

- Install Node.js 22.12+ and the .NET 10 SDK.
- Run `npm install`, `npm run check`, `npm run build`, and `npm test`.
- Run `npm start` and open `http://127.0.0.1:4173/`. For hot reload, also run `npm run dev` and open port 5173.

Code flow

- The `npm` scripts select the .NET SDK and build Vite assets into `frontend/dist`.
- `Program.cs` registers services and the system clock, initializes SQLite, and maps API endpoints.
- ASP.NET Core serves the built frontend; Vite development mode proxies `/api` to port 4173.

Data and rules

`DatabasePath` selects the SQLite file. `ProjectRoot` can override the repository root.

The project uses only the ASP.NET Core API and React frontend. No separate backend or legacy browser runtime is required.

Source blocks

`package.json` / lines 5-15 / "scripts"

`backend/dotnet/Program.cs` / lines 4-28 / var builder

`vite.config.ts` / lines 3-7 / export default

04. Create only missing tables and master data

Startup reuses the existing database and seeds missing definitions.

User actions

- Keep the existing `sql/onboarding.db` to retain your records.
- For a separate environment, set `DatabasePath` to a different file before starting.

Code flow

- `DatabaseInitializer` applies `CREATE TABLE IF NOT EXISTS` from `schema.sql`.
- `INSERT OR IGNORE` seeds roles, status/stage mappings, locations, segments, holidays, and the seven-day SLA.
- Demo users are created only when their usernames are missing. Initialization runs inside a transaction.

Data and rules

`role_master`, `status_master`, `location_master`, `segment_master`, `holiday_master`, `settings`, `users`.

Existing password hashes are retained. Replace demo credentials before shared deployment. Removing the database removes its business records.

Source blocks

`backend/dotnet/DatabaseInitializer.cs` / lines 8-47 / public static void Initialize

`backend/dotnet/Database.cs` / lines 10-20 / public SqlConnection Open

Enter the workspace

05. Open the app and restore an existing session

The boot screen waits for `/api/auth/me` before choosing login or workspace.

User actions

- Open or refresh the app.
- If your session is still valid, the workspace appears. Otherwise, sign in.

Code flow

- `useWorkspace` calls `/api/auth/me` on mount and stops applying the result after unmount.
- The API reads the session cookie, hashes the token, and looks up an active user.
- Expired session rows are removed. A missing or invalid session returns 401.

Data and rules

The cookie carries a random token. sessions stores only its SHA-256 hash, expiry, and user ID.

An expected 401 on first launch means there is no signed-in session; it is not a broken application.

Source blocks

```
frontend/useWorkspace.ts / lines 77-101 / useEffect() => {
backend/dotnet/AuthenticationService.cs / lines 30-42 / public User? SessionUser
backend/dotnet/ApiSessionMiddleware.cs / lines 7-61 / public static void UseApiSession
```

06. Sign in with a username and password

The server checks credentials and sets a protected twelve-hour cookie.

User actions

- Enter your username and password and choose Sign in.
- Correct active credentials open MIS Overview. Incorrect credentials show an error.

Code flow

- `LoginScreen` submits credentials to `useWorkspace.signIn` and POST `/api/auth/login`.
- `AuthenticationService` retrieves the existing salt and recalculates PBKDF2-HMAC-SHA256 with 200,000 iterations.
- A constant-time comparison verifies the hash. A random token is stored as a SHA-256 hash and returned through an `HttpOnly`, `SameSite Strict` cookie.

Data and rules

`users.password_hash / password_salt -> sessions.token_hash`. The login JSON contains identity and role, not the session token.

HTTPS requests get Secure cookies. The Python hash/session formats remain compatible with C#.

Source blocks

```
frontend/components/LoginScreen.tsx / lines 14-26 / async function submit
backend/dotnet/AuthenticationService.cs / lines 13-28 / public (User User, string Token)? Authenticate
backend/dotnet/PasswordHasher.cs / lines 8-9 / public static byte[] Digest
backend/dotnet/ApiEndpoints.cs / lines 11-22 / app.MapPost("/api/auth/login"
```

07. Your role controls what you can see and write

UI controls provide guidance; server policies enforce access.

User actions

- Admin manages users and all records. Operations manages case entry, edits, bulk upload, audit, and MIS.
- CSE works on own cases and request/resubmission/resolution statuses. MOFSL works on Stage 5/6 cases and MOFSL statuses.
- Viewer reads reports, the register, and audit history. RM is metadata, not a login role.

Code flow

- App filters navigation and CaseEntry filters status choices.
- ApiSessionMiddleware requires a session and restricts /api/users to admin.
- CaseAccessPolicy checks write status/ownership and filters read rows. The server sets CSE identity and role-specific owners itself.

Data and rules

Identity comes from the server-side session. Client-supplied changedBy or owner cannot bypass role checks.

MOFSL read visibility is based on the row owner or Stage 5/6; write permission still requires an existing Stage 5/6 context.

Source blocks

```
backend/dotnet/CaseAccessPolicy.cs / lines 7-12 / public static List<Dictionary<string, object?>> Visible
backend/dotnet/CaseAccessPolicy.cs / lines 14-39 / public static List<string> AccessErrors
frontend/app.tsx / lines 88-96 / const canWrite
backend/dotnet/CaseService.cs / lines 94-106 / private static Dictionary<string, object?> PreparePayload
```

08. Load the workspace without mixing old responses

Metadata, reports, entries, and history load together.

User actions

- Wait for the initial MIS snapshot, then choose a screen.
- Refresh the workspace or change report filters when needed.

Code flow

- useWorkspace.reload loads metadata, filtered MIS, register entries, and audit concurrently. Admin also loads users.
- A separate unfiltered report supplies register metrics when the MIS date range is filtered.
- A request-version counter prevents an older response from replacing a newer filter result.

Data and rules

/api/meta + /api/reports + /api/cases + /api/history + admin-only /api/users.

MIS date filters apply to reports. The register retains all visible entries and their unfiltered case metrics.

Source blocks

```
frontend/useWorkspace.ts / lines 47-75 / const reload =
```

Follow the six stages

09. Stage 1 / Receive and check the request

CSE sends the request; Operations checks whether information is complete.

User actions

- Create a New entry with Request Received from CSE.
- Operations can record Under Review by Operations.
- If information is missing, Operations records Query Raised to CSE - Missing Information with details. CSE records Resubmitted by CSE after resolving it.

Code flow

- New saves generate a Reference ID. A query-start event increments query count and makes the case NRFT.
- Resubmitted by CSE closes the earliest pending stage1 query interval.
- CSE-related entries retain the Reference ID and are owned by Operations.

Data and rules

Example journey: Request Received -> Query Raised -> Resubmitted. The stage remains Stage 1.

Query hold uses status-event timestamps, not the manually supplied resubmission date.

Source blocks

backend/dotnet/Domain.cs / lines 21-22 / new("Request Received from CSE"

backend/dotnet/CaseService.cs / lines 108-118 / private static string GenerateReferenceId

backend/dotnet/Domain.cs / lines 112-134 / public static (int Hold, int Queries) HoldAndQueries

10. Stage 2 / Prepare and send the form

Prepare the application form and send it through the chosen channel.

User actions

- Record Application Form Under Preparation.
- For Physical channel, record Physical Form Submitted to CSE.
- For Digital channel, record Digital Form Sent to Client. Enter the outward date manually unless admin opts into auto-capture.

Code flow

- Status Master derives Stage 2 from the selected status.
- CaseValidator rejects physical submission with a Digital channel and digital sending with a Physical channel.
- Admin auto-capture fills a blank outwardDate for either sending status.

Data and rules

cases.channel + case_entries.outward_date + status_history new status.

Preparation/sending statuses do not themselves raise query count.

Source blocks

backend/dotnet/Domain.cs / lines 23-23 / new("Application Form Under Preparation"

backend/dotnet/CaseValidator.cs / lines 9-69 / public static List<string> Validate

backend/dotnet/Domain.cs / lines 43-54 / public static readonly Dictionary<string, string> AutoDates

11. Stage 3 / Receive and review the signed form

Review the returned form and record discrepancies when necessary.

User actions

- Record Signed Form Received from CSE, then Under Review - Signed Form.
- If there is a problem, record Discrepancy Raised to CSE or Form Returned to CSE and explain it.

Code flow

- Each changed status creates an audit event.
- Discrepancy and return events start stage3 query intervals, each counted as a qualifying query.
- Any qualifying query makes the case NRFT, including when it was previously RFT.

Data and rules

Signed form date is manual by default. Query details are mandatory for discrepancy/return statuses.

Multiple query-start events are counted individually; one resolution closes the earliest matching interval.

Source blocks

backend/dotnet/Domain.cs / lines 24-25 / new("Signed Form Received from CSE"

backend/dotnet/CaseService.cs / lines 245-253 / private static void AppendStatusHistory

backend/dotnet/Domain.cs / lines 112-134 / public static (int Hold, int Queries) HoldAndQueries

12. Stage 4 / Resolve discrepancies and prepare submission

CSE provides the correction; Operations confirms readiness.

User actions

- CSE uses Add update and records Discrepancy Resolution Received or Resubmitted Form Received - Under Review.
- Operations checks the response and records Form Found in Order - Ready for MOFSL Submission.

Code flow

- CSE creates a related row rather than editing an earlier operational entry.
- Resolution statuses close the earliest pending stage3 hold interval.
- The readiness status maps to Stage 4 and prepares the operational handoff.

Data and rules

The same Reference ID groups the original and resolution entries.

Resolving the discrepancy ends that hold interval but does not change NRFT back to RFT.

Source blocks

frontend/components/CaseEntry.tsx / lines 32-45 / function initialEntry
backend/dotnet/Domain.cs / lines 26-26 / new("Discrepancy Resolution Received")
backend/dotnet/CaseAccessPolicy.cs / lines 14-39 / public static List<string> AccessErrors

13. Stage 5 / Submit to MOFSL and resolve queries

Operations submits the form; MOFSL can record its query or resolution.

User actions

- Operations records Submitted to MOFSL and sets owner to MOFSL.
- MOFSL uses Add update to record Query Raised by MOFSL with details.
- When resolved, MOFSL records Query Resolved - Resubmitted to MOFSL.

Code flow

- MOFSL writes require the selected existing entry or latest related-entry context to have reached Stage 5/6.
- The API forces owner to MOFSL for MOFSL users.
- A stage5 query starts a hold interval; its resolution closes the earliest matching interval.

Data and rules

submittedDate describes the operational submission date. Audit timestamps drive query calculations.

A MOFSL user cannot create a brand-new case or update an earlier-stage case by submitting a forged payload.

Source blocks

backend/dotnet/Domain.cs / lines 27-28 / new("Submitted to MOFSL")
backend/dotnet/CaseAccessPolicy.cs / lines 14-39 / public static List<string> AccessErrors
backend/dotnet/CaseService.cs / lines 94-106 / private static Dictionary<string, object?> PreparePayload

14. Stage 6 / Open the account and close communication

Record account details and complete the case.

User actions

- Enter an account number and account-opening date, then record Account Opened.
- After communication, record Communication Sent - Case Closed.

Code flow

- Both statuses are classified as closed and map to Stage 6.
- Validation requires an account number and valid opening date. The opening date cannot precede the inward date.
- MIS stops gross TAT at the opening date, shows zero aging for closed status, and retains full query history.

Data and rules

case_entries.account_number / account_opening_date; closed count in MIS.

NRFT stays NRFT after closure. Admin can auto-fill a blank opening date on Account Opened, using the server clock.

Source blocks

```
backend/dotnet/Domain.cs / lines 29-29 / new("Account Opened")
backend/dotnet/CaseValidator.cs / lines 9-69 / public static List<string> Validate
backend/dotnet/ReportService.cs / lines 139-184 / private static void AddMetrics
```

15. Exceptions / Hold, reject, or cancel

Exceptions remain visible; rejected/cancelled cases leave the performance base.

User actions

- Operations/Admin records On Hold, Rejected, or Cancelled by Client as appropriate.
- Management reviews these counts alongside workload and quality.

Code flow

- All three statuses map to Exception.
- On Hold remains in the eligible RFT/TAT base and appears in attention cases.
- Rejected and Cancelled remain total/exception counts but are excluded from RFT percentages and average TAT.

Data and rules

Exception pipeline includes On Hold. Performance exclusions include only Rejected and Cancelled by Client.

On Hold alone does not start a query-hold interval. Only qualifying query/discrepancy events do. The API maps stages but does not enforce a full Stage 1 -> 6 transition graph.

Source blocks

```
backend/dotnet/Domain.cs / lines 17-17 / public static readonly string[] Excluded
backend/dotnet/ReportService.cs / lines 16-22 / private static bool Eligible
frontend/components/ReportingViews.tsx / lines 308-430 / export function Dashboard
```

Trace one save

16. Step 1 / Collect and validate the React form

CaseEntry owns editable state and submits POST or PUT.

User actions

- Choose New Entry or Edit/Add update from the register.
- Fill required fields, choose a permitted status, and choose Save entry.

Code flow

- CaseEntry starts with defaults or a selected row. CSE/MOFSL prepares a related update; Admin/Operations edits the selected entry.
- Frontend validation checks required fields, PAN, masters, dates, channel consistency, query details, and known related references.
- POST /api/cases creates an entry. PUT /api/cases/{id} updates an entry.

Data and rules

React form state -> JSON payload. The New Reference ID field is read-only until the server generates it.

Frontend validation gives quick feedback; direct API callers still face all backend checks.

Source blocks

frontend/components/CaseEntry.tsx / lines 73-92 / async function save

frontend/lib.ts / lines 168-251 / export function validate

17. Step 2 / Authenticate and route the API request

Middleware resolves identity before the endpoint invokes the case service.

User actions

- A valid save continues. An expired session returns to login.
- Malformed JSON is rejected before business data is changed.

Code flow

- api() sends same-origin cookies and JSON.
- ApiSessionMiddleware resolves the user from the session; ApiEndpoints.Payload accepts a JSON object with scalar fields.
- The endpoint calls CaseService.SaveCase with the server-resolved user and returns its response body/status.

Data and rules

HTTP -> middleware -> Current(context) + Payload(context) -> SaveCase.

Missing sessions produce 401. Case permission failures produce 403. Invalid data produces 400.

Source blocks

frontend/lib.ts / lines 112-125 / export async function api

backend/dotnet/ApiSessionMiddleware.cs / lines 7-61 / public static void UseApiSession

backend/dotnet/ApiEndpoints.cs / lines 41-50 / app.MapPost("/api/cases"

backend/dotnet/ApiEndpoints.cs / lines 74-86 / private static async Task<Dictionary<string, object?>> Payload

18. Step 3 / Normalize, authorize, then validate

The service prepares trusted values before any persistent writes.

User actions

- Review field errors and correct the form if the server rejects it.
- For related entries, use an existing Reference ID.

Code flow

- PreparePayload trims strings, uppercases PAN, normalizes valid DD-MM-YYYY dates, and applies role-specific CSE/owner values.
- CaseAccessPolicy checks permitted statuses, existing ownership, and stage context.
- CaseValidator verifies masters, controlled values, date validity, closed/query requirements, and original-case existence.

Data and rules

Normalized payload + existing case context + authenticated User.

PAN must match five letters, four digits, and a final letter. Invalid dates such as 31 September are rejected.

Source blocks

```
backend/dotnet/CaseService.cs / lines 49-92 / public (int Status, object Body) SaveCase  
backend/dotnet/CaseValidator.cs / lines 9-69 / public static List<string> Validate  
backend/dotnet/Domain.cs / lines 79-99 / public static Dictionary<string, object?> Normalize
```

19. Step 4 / Save the case and entry in one transaction

Client metadata, entry details, and status history succeed together.

User actions

- After a successful save, the register reloads and displays the generated Reference ID.
- An invalid or denied request leaves the previous records intact.

Code flow

- ResolveCase finds the existing case/entry or creates the original case.
- UpdateClientDetails updates shared metadata. WriteEntry inserts a related entry or updates the selected row.
- AppendStatusHistory adds a changed-status event. The service reads the saved row and commits once. Without a commit, the transaction rolls back.

Data and rules

cases + case_entries + status_history in one SQLite transaction.

Editing cannot change the stable Reference ID. SQL values are bound as parameters rather than concatenated from form input.

Source blocks

```
backend/dotnet/CaseService.cs / lines 49-92 / public (int Status, object Body) SaveCase  
backend/dotnet/CaseService.cs / lines 207-243 / private static long WriteEntry  
backend/dotnet/Database.cs / lines 24-39 / private static SqlCommand Command
```

20. Step 5 / Append status history with the real actor

History records who changed status and supplies the query-calculation timeline.

User actions

- Open Audit History to see the old status, new status, actor, owner, timestamp, and notes.
- A change to remarks without a status change does not add another status-history event.

Code flow

- For a new related entry, the previous status comes from the latest existing entry.
- AppendStatusHistory compares old/new status IDs and inserts only when they differ.
- changed_by uses User.DisplayName from the authenticated session, even if the payload includes a different name.

Data and rules

status_history is appended through the case service; there is no API to edit or delete history.

This is status-change audit history, not a separate audit event for every field edit.

Source blocks

backend/dotnet/CaseService.cs / lines 245-253 / private static void AppendStatusHistory

backend/dotnet/CaseService.cs / lines 38-47 / public object GetHistory

frontend/components/ReportingViews.tsx / lines 262-306 / export function AuditHistory

21. Manual dates and admin-only automatic capture

Operational dates are supplied by people unless an admin opts in.

User actions

- Enter the generic dates and the dedicated dates under Stage dates.
- For a backdated status change, set Status business date. Admin can fill blank status-related dates automatically.

Code flow

- EntryFields defines the persisted stage date columns. CaseValidator validates them.
- The API captures a business date for each status event separately from its audit recording timestamp.
- Admin automatic capture fills blank generic and dedicated status dates without replacing supplied dates.

Data and rules

Manual process-date fields and audit event timestamps are separate.

An explicit Status business date takes precedence, then the dedicated date for that status. Undated events fall back to audit dates and are marked in Case Details.

Source blocks

backend/dotnet/CaseService.cs / lines 94-106 / private static Dictionary<string, object?> PreparePayload

backend/dotnet/Domain.cs / lines 79-99 / public static Dictionary<string, object?> Normalize

backend/dotnet/Domain.cs / lines 43-54 / public static readonly Dictionary<string, string> AutoDates

backend/dotnet/EntryFields.cs / lines 4-5 / public static class EntryFields

backend/dotnet/CaseService.cs / lines 255-265 / private static object? BusinessDate

22. Count operational touches automatically

A touch is one successful case save.

User actions

- Save a new entry or edit an existing entry. Read Touch Count in the register or Case Details.
- A same-status edit counts as a touch. Viewing records does not.

Code flow

- UpdateClientDetails increments the case counter inside the same transaction as entry persistence.
- Denied or invalid saves roll back; a client-supplied Touch Count is ignored.
- Migration initializes older cases from their saved entry count once. Subsequent saves increment automatically.

Data and rules

cases.touch_count is shared by every entry under the Reference ID.

Existing entry count is a minimum legacy baseline. Earlier edits were not tracked and cannot be reconstructed reliably.

Source blocks

backend/dotnet/CaseService.cs / lines 191-205 / private static void UpdateClientDetails

backend/dotnet/DatabaseInitializer.cs / lines 8-47 / public static void Initialize

backend/tests/ApiTests.cs / lines 63-83 / public void TouchCountIsAutomaticAtomicAndCountsSameStatusSaves

Understand the MIS

23. Choose one current row per case

The register has entries; MIS selects the latest entry for each case.

User actions

- Apply an inward date range on MIS Overview.
- Use the register to view the individual entries behind a case.

Code flow

- CaseQueries orders entries by updated_at descending, then ID descending.
- ReportService.LatestCases groups by caseId and takes the first row.
- It finds the earliest valid inward date across the group, applies filters to that original date, and records entry count minus one as resubmissionCount.

Data and rules

Latest entry determines current status/stage. Earliest inward date determines the case TAT start.

The field named resubmissionCount counts all additional entries, including modifications and discrepancy resolutions.

Source blocks

backend/dotnet/ReportService.cs / lines 112-137 / private static List<Dictionary<string, object?>> LatestCases

backend/dotnet/CaseQueries.cs / lines 21-26 / public static List<Dictionary<string, object?>> CaseRows

24. Calculate query count and query hold days

Status-history events determine quality and waiting time.

User actions

- Raise queries with the appropriate status and details.
- Record the matching resolution status when the response arrives.

Code flow

- Events are ordered by business date and event ID; older undated events use the audit date.
- Each query-start event enters a queue for stage1, stage3, or stage5 and increments query count.
- A matching resolution removes the earliest pending start. Working days in each interval contribute to hold; unresolved starts count through today.

Data and rules

RFT when queryCount is zero; NRFT when queryCount is positive.

Example: a Friday query resolved on Tuesday counts Monday/Tuesday, excluding listed holidays. Overlapping query intervals are counted individually, not merged.

Source blocks

backend/dotnet/Domain.cs / lines 112-134 / public static (int Hold, int Queries) HoldAndQueries

backend/dotnet/ReportService.cs / lines 139-184 / private static void AddMetrics

25. Calculate working-day TAT and SLA

TAT excludes weekends and Holiday Master dates; net TAT also subtracts query hold.

User actions

- Read gross/net TAT and SLA breaches in MIS.
- Maintain correct inward/opening dates because those dates affect reporting.

Code flow

- WorkingDays excludes the start day and includes eligible days through the end day.
- Gross TAT runs from original inward date to opening date for closed cases, or today for open cases.
- Net TAT = $\max(0, \text{gross TAT} - \text{query hold})$. SLA breach uses gross TAT > the configured SLA, seven working days by default.

Data and rules

Example: Friday 23 Jan 2026 -> Tuesday 27 Jan, with Monday a holiday, is one working day.

SLA uses gross TAT, not net TAT. Rejected/cancelled cases leave the average-TAT base.

Source blocks

backend/dotnet/Domain.cs / lines 101-110 / public static int WorkingDays
 backend/dotnet/ReportService.cs / lines 139-184 / private static void AddMetrics
 backend/tests/ApiTests.cs / lines 436-474 / public void HistoryCalculationsAndExclusions

26. Measure gross TAT and SLA within each stage

Stage intervals accumulate across repeated visits.

User actions

- Open Case Details for all six stage timings and breach flags.
- Use SLA Settings as Admin to change overall and per-stage limits.

Code flow

- StageMetrics orders business-date events and supplied milestone dates.
- Each interval belongs to the starting stage, with Monday-Friday working days excluding holidays. Repeated visits accumulate.
- A stage breach is gross stage TAT greater than its threshold, default 7 days. A skipped stage stays unknown.

Data and rules

stageTat + stageSlaBreaches + stageSlaDays in report responses.

Gross stage TAT includes query waiting time. Legacy events may use audit dates. Case aging and time in a stage are displayed separately.

Source blocks

backend/dotnet/StageMetrics.cs / lines 8-38 / public static Dictionary<string, int?> Calculate
 backend/dotnet/ReportService.cs / lines 139-184 / private static void AddMetrics
 frontend/components/CaseDetails.tsx / lines 4-83 / export function CaseDetails

27. Read the overview and attention list

The API returns summary, pipeline, groups, and masked case details.

User actions

- Review open/closed volume, RFT, average TAT, breaches, and exceptions.
- Use the stage pipeline to assess current workload and aging.
- Review up to ten open cases above SLA or currently On Hold.

Code flow

- ReportService aggregates eligible cases and returns the full reporting response.
- Dashboard renders numbers and bar widths; it does not recalculate business metrics.
- Finished/rejected/cancelled cases are excluded from the attention list.

Data and rules

summary + pipeline + groups + cases. PAN is removed and replaced by panMasked before report serialization.

Average aging uses zero for closed cases. Pipeline includes Stage 6 and Exception.

Source blocks

backend/dotnet/ReportService.cs / lines 23-110 / public object Report
frontend/components/ReportingViews.tsx / lines 308-430 / export function Dashboard

28. Compare CSE, location, and segment

Performance MIS changes the grouping dimension without changing the calculation rules.

User actions

- Choose CSE, Location, or Vertical / Segment.
- Compare totals, RFT/NRFT, net TAT, overall/stage SLA cases, and Stage 1/3/5 query attribution.

Code flow

- React updates dimension and triggers a workspace reload.
- The API permits cseName, location, or segment and defaults unsupported dimensions to cseName.
- Groups use the same rejection/cancellation exclusions; missing grouping values appear as Unassigned.

Data and rules

GET /api/reports?dimension=location&start=YYYY-MM-DD&end=YYYY-MM-DD.

All groups are limited by your role visibility and the selected original-inward date range.

Source blocks

frontend/components/ReportingViews.tsx / lines 65-147 / export function Performance
backend/dotnet/ReportService.cs / lines 23-110 / public object Report

29. Search the register and start an edit or update

Each displayed row is an entry, with case-level metrics attached for context.

User actions

- Search a reference/client or filter by status.
- Admin/Operations chooses Edit. CSE/MOFSL chooses Add update. Viewer has no write action.
- Choose Details for original inward, resubmission count, automatic touches, stage timing, SLA, and all process dates.

Code flow

- CaseRegister filters the visible entries locally.
- It joins case metrics by caseId from the unfiltered reporting snapshot.
- App opens CaseEntry with the selected row; initialEntry decides whether it is an exact edit or a related update.

Data and rules

PAN is masked on screen. Viewer case responses omit raw PAN; permitted operational case responses can include PAN for editing.

Register query/hold/TAT fields describe the current case, not only the row's own entry status.

Source blocks

frontend/components/ReportingViews.tsx / lines 149-260 / export function CaseRegister

frontend/components/CaseEntry.tsx / lines 32-45 / function initialEntry

backend/dotnet/CaseService.cs / lines 23-36 / public object GetCases

frontend/components/CaseDetails.tsx / lines 4-83 / export function CaseDetails

Upload and administer

30. Preview and import a CSV

Rows are validated locally, then each valid row uses the ordinary save API.

User actions

- Download the template, fill it, and choose the CSV file.
- Inspect each ready/error row, then choose Import valid rows.
- Review Imported markers and any server failures.

Code flow

- csvRecords supports a BOM, quoted commas, escaped quotes, and multiline values.
- parseCsv maps headers, applies defaults, normalizes DD-MM-YYYY dates, and validates against known references.
- BulkUpload sends rows sequentially, marks successes, and keeps failures visible. It stops on session expiry.

Data and rules

Every CSV row uses POST /api/cases and the same permissions, validation, transaction, and audit rules as a form save.

Bulk upload is not atomic. Earlier successes remain saved if a later row fails. Successful rows cannot be imported twice in the same preview.

Source blocks

frontend/lib.ts / lines 252-306 / export function csvRecords

frontend/components/BulkUpload.tsx / lines 43-66 / async function importValid

31. Create an application user

Admin assigns a role and an initial password.

User actions

- Open User Administration.
- Supply a unique username, display name, role, and password of at least eight characters.
- Choose Create user and review the refreshed user list.

Code flow

- Middleware rejects non-admin calls to /api/users.
- UserService validates identity fields and the role, generates a fresh salt, and stores a PBKDF2 hash.
- Duplicate usernames return a 400 error; the user list excludes password hashes and salts.

Data and rules

users.role_id -> role_master.id. Usernames are unique without case sensitivity.

This screen creates users. It does not currently provide password reset, disabling, or role-edit controls.

Source blocks

frontend/components/UserAdministration.tsx / lines 11-26 / async function createUser

backend/dotnet/UserService.cs / lines 11-25 / public static long CreateUser

backend/dotnet/UserService.cs / lines 27-39 / public (int, object) AddUser

32. Configure overall and stage SLA as Admin

SLA limits are persisted centrally.

User actions

- Open SLA Settings and enter the overall and six stage thresholds in working days.
- Save settings to refresh reports.

Code flow

- GET and PUT /api/settings are protected by the API session middleware.
- SettingsService accepts only whole numbers from 1 to 3650 and saves all limits in one transaction.
- Stage limits fall back to the overall SLA when they have not been configured separately.

Data and rules

settings.sla_days and sla_stage1_days through sla_stage6_days.

Other roles cannot read or change administrative settings through the API.

Source blocks

backend/dotnet/SettingsService.cs / lines 6-7 / public sealed class SettingsService

backend/dotnet/ApiSessionMiddleware.cs / lines 7-61 / public static void UseApiSession

frontend/components/SlaSettings.tsx / lines 5-80 / export function SlaSettings

33. Sign out or recover from session expiry

The server invalidates the session and React clears protected snapshots.

User actions

- Choose Sign out when finished.
- If a request returns session-expired feedback, sign in again before continuing.

Code flow

- POST /api/auth/logout deletes the token hash and expires the cookie.
- useWorkspace.clearSession clears users, case rows, history, reports, and metadata.
- 401 errors trigger the same clearing behavior and a login message. Request-version invalidation prevents stale data returning after sign-out.

Data and rules

The session row is deleted; business cases and history remain.

If sign-out fails because the server is unreachable, the UI shows the error rather than falsely claiming the server session was deleted.

Source blocks

frontend/useWorkspace.ts / lines 23-45 / const clearSession

frontend/useWorkspace.ts / lines 114-118 / async function signOut

backend/dotnet/AuthenticationService.cs / lines 44-50 / public void Logout

Read and maintain the code

34. S / Give each module one clear responsibility

Separate HTTP, authentication, case policy, persistence, and reporting.

User actions

- When debugging a screen, begin with its React component.
- When a calculation is wrong, begin with ReportService/Domain. When permission is wrong, begin with CaseAccessPolicy.

Code flow

- App coordinates navigation. useWorkspace owns session and read snapshots. CaseEntry, BulkUpload, and UserAdministration own editable local state.
- Program composes dependencies; ApiEndpoints and middleware handle HTTP. Focused services implement distinct use cases.
- CaseService.SaveCase reads as a sequence of named steps inside one transaction.

Data and rules

AuthenticationService, UserService, CaseService, ReportService, CaseAccessPolicy, CaseValidator, CaseQueries, Database, DatabaseInitializer.

Shared JSX primitives handle tables, panels, statuses, and errors. JSON dictionary rows deliberately preserve the legacy API contract.

Source blocks

backend/dotnet/CaseService.cs / lines 49-92 / public (int Status, object Body) SaveCase

frontend/app.tsx / lines 27-240 / function App()

frontend/components/shared.tsx / lines 30-63 / export function Table

35. O / Put ordinary variation in data and small modules

Status mappings and masters feed generic reporting and display code.

User actions

- Read Status Master and the mapping definitions before changing workflow behavior.
- Change a specific policy or calculation module rather than duplicating rules in each screen.

Code flow

- Stages and query-start/end mappings are persisted in Status Master and consumed by generic reporting.
- Locations, segments, holidays, and SLA are read from the database.
- Role authorization and automatic-date mappings remain explicit, centralized code.

Data and rules

A new holiday requires data, not a new WorkingDays implementation.

This is a practical open/closed design, not a plugin framework. Adding a new role or special status may still require policy, validation, UI, and test changes.

Source blocks

backend/dotnet/ReportService.cs / lines 139-184 / private static void AddMetrics

backend/dotnet/CaseService.cs / lines 11-21 / public object GetMeta

backend/dotnet/Domain.cs / lines 101-110 / public static int WorkingDays

36. L / Keep replacements compatible with their contracts

Use composition, and prove the real clock substitution behaves correctly.

User actions

- Preserve the documented routes and JSON fields during refactoring.
- Run compatibility and role tests after changing authentication or saves.

Code flow

- Services are sealed and composed instead of creating an unnecessary service-inheritance hierarchy.
- TimeProvider.System and the test FixedClock satisfy the same time-provider contract.
- Integration tests verify password/session compatibility, status codes, and business outcomes through the real API.

Data and rules

Tests substitute time and isolated database files while exercising the same service/endpoint flow.

Liskov substitution concerns behavioral compatibility when replacing a dependency. It does not require every class to inherit from a base class.

Source blocks

backend/tests/ApiTests.cs / lines 17-211 / private sealed class FixedClock

backend/tests/ApiTests.cs / lines 494-511 / public void ExistingPythonPasswordAndSessionFormatsRemainValid

37. I / Keep caller contracts focused

Routes and screens receive the dependencies and callbacks they actually use.

User actions

- Inspect a component's Props to understand what its parent must provide.
- Inspect a route's injected service to identify its backend responsibility.

Code flow

- A case endpoint requests CaseService; authentication endpoints request AuthenticationService; report endpoints request ReportService.
- CaseEntry receives case data and onSave/onError callbacks. It does not receive user-administration behavior.
- The shared UI primitives accept small presentation props and do not call APIs themselves.

Data and rules

Focused component Props and service methods are the contracts used by callers.

TypeScript Props and C# service methods provide focused contracts without adding an interface wrapper for every concrete class.

Source blocks

frontend/components/CaseEntry.tsx / lines 22-30 / type Props
backend/dotnet/ApiEndpoints.cs / lines 33-68 / app.MapGet("/api/meta")
frontend/components/shared.tsx / lines 65-87 / export function Panel

38. D / Inject dependencies and abstract real variation

The composition root supplies storage and an abstract clock to services.

User actions

- Set DatabasePath for a different environment; do not hard-code it inside business operations.
- Use a fixed clock when testing dates, aging, and session expiry.

Code flow

- Program registers Database and TimeProvider.System through dependency injection.
- AuthenticationService, CaseService, and ReportService receive the clock through their constructors.
- Pure working-day and query helpers accept dates and holiday sets, independent of HTTP or filesystem state.

Data and rules

Production TimeProvider.System -> service; test FixedClock -> the same service.

Storage intentionally remains a concrete SQLite adapter. Replacing SQLite would require changes to the SQL/query layer. No claim is made that every dependency is abstracted.

Source blocks

backend/dotnet/Program.cs / lines 8-14 / builder.Services.AddSingleton
backend/dotnet/AuthenticationService.cs / lines 8-28 / public sealed class AuthenticationService
backend/tests/ApiTests.cs / lines 193-211 / public void BusinessDatesAndSessionExpiryUseTheInjectedClock

39. Verify behavior and keep the guide current

Automated checks protect the workflow and regenerate exact code references.

User actions

- Run `npm run check`, `npm run build`, and `npm test` before completing changes.
- Run `npm run format` for consistent source formatting.
- Run `npm run guide` after a code change, or let `npm run build` regenerate this HTML.

Code flow

- Frontend tests check CSV, related references, dates, and API errors.
- Backend tests exercise all roles, six-stage completion, query calculations, validation, audit, authentication, session expiry, compatibility, and injected time.
- The guide generator extracts source excerpts and line ranges from the working tree. It fails if a requested code marker disappears.

Data and rules

Tests use isolated temporary databases. This guide contains source snapshots, not business records.

The guide explains intended user order, while documenting actual enforcement and limitations. Code excerpts are labeled with path, line range, and a short source hash.

Source blocks

`package.json` / lines 5-15 / "scripts"

`backend/tests/ApiTests.cs` / lines 514-546 / `public void SixStageWorkflowCompletesWithAllQueryTypes`

`scripts/frontend-tests.mjs` / lines 30-43 / `test('CSV`

Verification and limits

Verified application

- npm run check and npm run build pass. The .NET build has zero warnings and errors.
- All 22 tests pass: 16 backend and 6 frontend.
- npm run guide:check confirms 39 segments with current source excerpts.
- Browser checks confirmed automatic touches, stage dates, Case Details, and configurable SLA breaches. All 39 guide cards opened 112 source tabs.

Business rules to remember

- Touch Count increments once per successful entry creation or edit. Reads and rejected saves do not count. Same-status saves leave Query Count unchanged.
- Stage SLA measures gross working days, including query waiting. Seven working days is the default. Admin configures overall and stage limits.
- Working days are Monday-Friday, excluding Holiday Master dates. Query holds reduce net TAT; stage gross TAT includes waiting.
- Rejected and cancelled cases remain exception counts but are excluded from final RFT percentage and average TAT.
- API authorization and audit identity are authoritative. Reporting views mask PAN.

Historical data and operating limits

- Legacy Touch Count uses saved entry count as a minimum baseline. Earlier edits cannot be reconstructed.
- Undated historical events use audit-date fallback for timing and are flagged in Case Details.
- Skipped stages remain unknown. No strict sequential transition graph is enforced.
- Bulk import is sequential and non-atomic. Earlier successful rows remain if a later row fails.
- The seeded holiday list covers 2026. No holiday-maintenance screen is provided.
- The Word document describes Excel, manual query holds, and an optional stage override. Later agreed instructions select this application, automatic holds, and status-derived stages.

Maintain the project

Keep HTTP handlers thin, business rules in focused services, and form state inside each screen. Update schema, additive migrations, backend tests, and AGENTS.md together for model changes. Regenerate the HTML guide after source edits.

```
npm run format
npm run check
npm run build
npm test
npm run guide:check
```

Reference documents

- documents/08-specification-audit.md
- documents/07-dotnet-react-migration.md
- documents/client-onboarding-flow.html
- AGENTS.md and README.md